Esercizi — strutture dati complesse e iteratori

Lezione coperta: `L07` — stringhe, tuple, liste e dizionari; indici e slicing; visitare una struttura con il `while` e con il `for`; `range`; le liste di liste

I cicli hanno la loro scheda, `L06_esercizi.md`, e qui si dà per saputa: il nuovo sono le **strutture**, cioè tanti valori sotto un nome solo, e il modo di raggiungerli uno per uno.

Come si lavora. Tre gradini, come per i cicli: prima **seguire** il codice e dire che cosa stampa, poi **trovare** l'errore, infine **scrivere**. Le soluzioni sono sotto ogni esercizio: aprile dopo aver provato.

Il primo gradino si fa con carta e penna, poi si controlla nell'interprete. Con le strutture la domanda da farsi è sempre la stessa: **che indice, o che chiave, sto usando?** Il primo elemento ha indice 0, l'ultimo ha indice `len(...) - 1`.

Prima di cominciare, leggi `L07_video.md`. I video sono registrati con Python 2, e in alcuni punti mostrano un comportamento che Python 3 non ha più: gli esercizi 7 e 9 partono proprio da due di quei punti.

Gradino 1 — seguire il codice

Esercizio 1 — Indici e slicing (difficoltà: •)

Che cosa stampa ciascuna riga?

```
parola = "Statistica"
print(parola[0])
print(parola[-1])
print(parola[3])
print(parola[2:5])
print(parola[:4])
print(parola[4:])
print(len(parola))
print(parola[len(parola) - 1])
```

Soluzione

Mostra la soluzione

```
S
a
t
ati
Stat
istica
10
a
```

Gli indici partono da 0: `S` è `parola[0]`, la prima `t` è `parola[1]`, e `parola[3]` è la seconda. Gli indici negativi contano dal fondo: `parola[-1]` è l'ultimo carattere.

Nello slicing `parola[i:j]` il secondo indice è escluso. `parola[2:5]` prende gli indici 2, 3 e 4, cioè tre caratteri: `j - i`. Per questo `parola[:4]` e `parola[4:]` si incastrano senza buchi e senza sovrapposizioni, e insieme ridanno la stringa intera.

L'ultimo carattere ha indice `len(parola) - 1`, cioè 9. `parola[10]` darebbe `IndexError`: è l'errore dell'esercizio 8.

Esercizio 2 — Che cosa produce `range` (difficoltà: •)

`list(...)` trasforma un `range` in una lista, così si vede che cosa contiene. Che cosa stampa ciascuna riga?

```
print(list(range(5)))
print(list(range(2, 6)))
print(list(range(1, 10, 3)))
print(list(range(10, 0, -3)))
print(list(range(5, 2)))
print(range(5))
```

Soluzione

Mostra la soluzione

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5]
[1, 4, 7]
[10, 7, 4, 1]
[]
range(0, 5)
```

`stop` è escluso, come il secondo indice dello slicing: `range(5)` arriva a 4, `range(2, 6)` a 5. Così `range(len(lista))` dà esattamente gli indici validi di una lista, da 0 a `len(lista) - 1`.

`range(1, 10, 3)` si ferma a 7 perché il passo successivo, 10, non è più minore di `stop`. Con un passo negativo si scende, e `stop` resta escluso: lo 0 non c'è.

`range(5, 2)` è **vuoto**, senza errori: con il passo 1 da 5 non si arriva mai sotto 2. Un `for` su un `range` vuoto fa zero giri.

L'ultima riga stampa `range(0, 5)` e non una lista: in Python 3 `range` è un oggetto suo, che produce i numeri uno alla volta. Il video `L07_f`, registrato con Python 2, mostra invece una lista.

Esercizio 3 — La lista copiata (difficoltà: •)

Che cosa stampano le quattro `print`?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
print(b)

c = list(a)
c.append(5)
print(a)
print(c)
```

Soluzione

Mostra la soluzione

```
[1, 2, 3, 4]
[1, 2, 3, 4]
[1, 2, 3, 4]
[1, 2, 3, 4, 5]
```

`b = a` non copia la lista: le dà un secondo nome. `a` e `b` sono la **stessa** lista, e quello che si fa attraverso `b` si vede attraverso `a`. Lo verifichi con `id(a) == id(b)`, che vale `True`: è il frame *Copia di liste* di `L07_d`.

`list(a)` costruisce invece una lista **nuova** con gli stessi elementi: `c` cresce, `a` no.

Con stringhe e tuple la domanda non si pone, perché non si possono modificare: nessuno può cambiarle da sotto i tuoi piedi.

Esercizio 4 — `append` o `extend` (difficoltà: •)

Che cosa stampano le quattro `print`? Per ciascuna, quanto vale `len`?

```
a = [1, 2]
a.append([3, 4])
print(a, len(a))

b = [1, 2]
b.extend([3, 4])
print(b, len(b))

c = [1, 2]
c.append('ciao')
print(c, len(c))
```

```
d = [1, 2]
d.extend('ciao')
print(d, len(d))
```

Soluzione

Mostra la soluzione

```
[1, 2, [3, 4]] 3
[1, 2, 3, 4] 4
[1, 2, 'ciao'] 3
[1, 2, 'c', 'i', 'a', 'o'] 6
```

`append` aggiunge un elemento solo, qualunque cosa sia. Se gli dai una lista, in coda finisce **la lista intera**, come un elemento: `a` ha tre elementi, e il terzo è `[3, 4]`. `a[2]` vale `[3, 4]`, e `a[2][0]` vale 3: è già una lista di liste, come in `L07_g`.

`extend` aggiunge uno per uno gli elementi di quello che gli dai. `b` ha quattro elementi, tutti numeri. È il frame di `extend` in `L07_d`, che allunga `serie` con `serie2`.

Le ultime due righe mostrano la differenza più sorprendente. `extend` scorre quello che riceve come farebbe un `for`, e un `for` su una stringa visita i **caratteri**: `'ciao'` entra come quattro lettere separate. Con `append` entra come una parola sola.

Per ricordarla: **`append` aggiunge un elemento; `extend` amplia la lista di tanti elementi quanti ne ha quello che riceve.**

Esercizio 5 — Il dizionario (difficoltà: •)

Che cosa stampa?

```
temperature = {'lun': 3, 'mar': -1}
temperature['mer'] = 2
temperature['mar'] = 0
print(len(temperature))
for giorno in temperature:
    print(giorno, temperature[giorno])
```

Soluzione

Mostra la soluzione

```
3
lun 3
mar 0
```

```
mer 2
```

Assegnare a una chiave nuova la aggiunge, assegnare a una chiave che c'è già ne cambia il valore. La riga è la stessa, `diz[chiave] = valore`: decide il dizionario, a seconda che la chiave ci sia o no. Per questo `len` vale 3 e non 4: `'mar'` non è stata aggiunta una seconda volta, è stata sovrascritta.

Il `for` visita le **chiavi**, nell'**ordine in cui sono state inserite**: `'mar'` resta al secondo posto anche dopo la modifica. Il video `L07_f` dice che l'ordine non è garantito: valeva per Python 2, non per Python 3.

Gradino 2 — trovare l'errore

Esercizio 6 — Cambiare una lettera (difficoltà: ••)

Il programma dovrebbe trasformare `"pane"` in `"cane"`. Che cosa succede?

```
parola = "pane"
parola[0] = "c"
print(parola)
```

Soluzione

Mostra la soluzione

```
TypeError: 'str' object does not support item assignment
```

Le stringhe sono immutabili: un carattere si può leggere con `parola[0]`, ma non sostituire. Si costruisce invece una stringa **nuova**, e la si assegna allo stesso nome:

```
parola = "pane"
parola = "c" + parola[1:]
print(parola)
```

Stampa `cane`. `parola[1:]` è `"ane"`, tutto tranne il primo carattere.

Con una lista la prima versione avrebbe funzionato: `l = ['p', 'a', 'n', 'e']` e poi `l[0] = 'c'` è lecito, perché le liste sono **mutabili**. È la differenza che la tabella delle proprietà di `L07_a` chiama *plasticità nel tempo*.

Esercizio 7 — Il `sort` che ordina male (difficoltà: ••)

Il programma legge quattro numeri e li stampa in ordine crescente. Con `12`, `9`, `100`, `25` dovrebbe stampare `[9, 12, 25, 100]`. Che cosa stampa, e perché?

```
numeri = []
letti = 0
while letti < 4:
    numeri.append(input("un numero: "))
    letti = letti + 1
numeri.sort()
print(numeri)
```

Soluzione

Mostra la soluzione

```
['100', '12', '25', '9']
```

`input()` **restituisce sempre una stringa**, e il `sort` ordina stringhe in ordine **alfabetico**, carattere per carattere: `'100'` viene prima di `'12'` perché al secondo carattere `'0'` viene prima di `'2'`, e `'9'` va in fondo perché `'9'` viene dopo `'1'` e `'2'`.

L'indizio è nella stampa: gli **apici**. Una lista di numeri si stampa senza.

La correzione è convertire **appena si legge**:

```
    numeri.append(int(input("un numero: ")))
```

Ora stampa `[9, 12, 25, 100]`.

È lo stesso errore dei frame di `append`, `extend` e `sort` nel video `L07_d`: lì non si vede, perché in Python 2 `input()` convertiva da solo. Una lista ordinata, nell'ordine sbagliato: nessun errore, nessun messaggio.

Esercizio 8 — Un giro di troppo (difficoltà: ••)

Il programma dovrebbe sommare gli elementi della lista. Che cosa succede?

```
lista = [4, 8, 15, 16]
somma = 0
indice = 0
while indice <= len(lista):
    somma = somma + lista[indice]
    indice = indice + 1
print(somma)
```

Soluzione

Mostra la soluzione

```
IndexError: list index out of range
```

La lista ha 4 elementi, con indici da 0 a **3**. Con `indice <= len(lista)` il ciclo fa anche il giro con `indice = 4`, e `lista[4]` non esiste.

La condizione giusta è `indice < len(lista)`. Stampa `43`.

È l'**errore di uno** di `L06`, l'esercizio 6, portato sugli indici: lì mancava l'ultimo giro, qui ce n'è uno in più. Quando non ti serve l'indice, il `for` lo evita del tutto, perché non lo scrivi tu:

```
somma = 0
for elemento in lista:
    somma = somma + elemento
print(somma)
```

Esercizio 9 — Il dizionario visitato con un indice (difficoltà: ••)

Il programma dovrebbe stampare i giorni di ciascun mese. Che cosa succede?

```
giorni = {'gennaio': 31, 'febbraio': 28, 'marzo': 31}
indice = 0
while indice < len(giorni):
    print(giorni[indice])
    indice = indice + 1
```

Soluzione

Mostra la soluzione

```
KeyError: 0
```

Un dizionario ha chiavi, non posizioni. `giorni[0]` cerca la chiave `0`, che non c'è: le chiavi sono `'gennaio'`, `'febbraio'` e `'marzo'`. È il primo frame del video `L07_f`, *Dizionari e visita dei loro elementi*.

La correzione naturale è il `for`, che visita le chiavi:

```
for mese in giorni:
    print(mese, giorni[mese])
```

Se ti serve proprio un indice, converti le chiavi in una lista con `list(giorni.keys())`, come fa il frame successivo dello stesso video:

```
chiavi = list(giorni.keys())
indice = 0
while indice < len(chiavi):
    print(chiavi[indice], giorni[chiavi[indice]])
    indice = indice + 1
```

Senza `list`, `giorni.keys()` restituisce una **vista**, e `vista[indice]` dà `TypeError`. Il video mostra `keys()` senza `list` perché in Python 2 restituiva già una lista.

Esercizio 10 — La tabella con un campo in più (difficoltà: ••)

Ogni riga è una persona con le spese di tre giorni, e il programma dovrebbe stampare il totale di ciascuna: Anna 50, Luca 50.

```
spese = [['Anna', 12, 30, 8],
         ['Luca', 25, 10, 15]]

for riga in spese:
    totale = 0
    colonna = 0
    while colonna < len(riga):
        totale = totale + riga[colonna]
        colonna = colonna + 1
    print(riga[0], totale)
```

Il programma si ferma con un errore. Un collega lo «corregge» cambiando una sola riga, `colonna = 2`: ora gira senza errori. Che cosa stampa? Qual è la correzione giusta?

Soluzione

Mostra la soluzione

La prima versione si ferma subito:

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Il ciclo parte dalla colonna 0, che contiene il **nome**, e `0 + 'Anna'` non si può fare.

La versione del collega stampa:

```
Anna 38
Luca 25
```

Gira, ma salta la colonna 1, cioè **la prima spesa** di ciascuno. La correzione giusta è `colonna = 1`: il nome occupa un solo campo, e le spese cominciano subito dopo.

```
Anna 50
Luca 50
```

La versione del collega è peggiore di quella con l'errore. L'errore si vede e ti ferma; il totale sbagliato prosegue e finisce in una relazione. È quello che succedeva alla tabella dei seggi della Florida nel video `L07_g`: un campo non dichiarato all'inizio di ogni riga, due programmi che giravano, e i voti di un candidato attribuiti a un altro.

Il controllo da fare **prima** di scrivere il ciclo: conta i campi della prima riga, e scrivi accanto a ciascuno che cosa contiene. Dove cominciano i numeri lo dice quel conteggio, non il ricordo.

Gradino 3 — scrivere

Lo schema è nei frame dei lucidi (per questi esercizi non c'è un video che li risolva), e la soluzione sotto ogni esercizio è **una** delle possibili.

Esercizio 11 — Una settimana di temperature (difficoltà: •••)

Le temperature minime di una settimana di gennaio a Milano, da lunedì a domenica:

```
temperature = [3, -1, -4, 0, 2, 5, -2]
```

Scrivi un programma che stampi la temperatura **minima** e la **massima** della settimana, la **media**, e **quanti giorni** sono stati sotto zero. Scrivi i calcoli con un ciclo.

Soluzione

Mostra la soluzione

```
temperature = [3, -1, -4, 0, 2, 5, -2]

minima = temperature[0]
massima = temperature[0]
somma = 0
sottoZero = 0
for temperatura in temperature:
    if temperatura < minima:
        minima = temperatura
    if temperatura > massima:
        massima = temperatura
    somma = somma + temperatura
    if temperatura < 0:
        sottoZero = sottoZero + 1

print("minima:", minima)
print("massima:", massima)
print("media:", somma / len(temperature))
print("giorni sotto zero:", sottoZero)
```

Stampa:

```
minima: -4
massima: 5
media: 0.42857142857142855
giorni sotto zero: 3
```

Minimo e massimo partono dal primo valore della lista, non da 0: con `massima = 0` funzionerebbe per caso, perché c'è un 5; con `minima = 0` funzionerebbe anche, perché ci sono temperature negative. Su una settimana tutta sopra zero `minima = 0` stamperebbe 0, una temperatura che non c'è mai stata. È la regola del massimo vista con i cicli: **si parte dal primo valore**.

Un ciclo solo basta per tutti e quattro i conti: ogni valore si guarda una volta e aggiorna quello che lo riguarda. `0` non è sotto zero, quindi i giorni sono 3: martedì, mercoledì, domenica.

Python ha già `min()`, `max()` e `sum()`, e `min(temperature)` dà -4. Usali per **controllare** il tuo ciclo, non al suo posto: il punto dell'esercizio è lo schema, che ti servirà quando il conto non è uno di quei tre.

Esercizio 12 — Contare le lettere (difficoltà: •••)

Scrivi un programma che conti quante volte compare ciascuna lettera nella frase `"la pioggia a milano"`, spazi esclusi, e stampi ogni lettera con il suo conteggio.

Un suggerimento: `chiave in diz` vale `True` se la chiave è già nel dizionario, `False` altrimenti.

Soluzione

Mostra la soluzione

```
frase = "la pioggia a milano"

conteggio = {}
for lettera in frase:
    if lettera != " ":
        if lettera in conteggio:
            conteggio[lettera] = conteggio[lettera] + 1
        else:
            conteggio[lettera] = 1

for lettera in conteggio:
    print(lettera, conteggio[lettera])
```

Stampa:

```
l 2
a 4
p 1
i 3
o 2
g 2
m 1
n 1
```

Il dizionario parte vuoto, e cresce una chiave alla volta. La prima volta che si incontra una lettera la chiave non c'è, e la si crea con il valore 1; le volte successive c'è, e si incrementa. Senza il controllo `if lettera in conteggio`, la prima `l` darebbe `KeyError: 'l'`: non si può incrementare un valore che non esiste.

Le lettere escono nell'ordine in cui sono comparse **la prima volta** nella frase, perché è l'ordine di inserimento delle chiavi.

È un dizionario e non una lista perché si raggiunge **per lettera**, non per posizione: con una lista dovresti decidere tu a quale indice va la `g`.

Esercizio 13 — La pioggia a Milano (difficoltà: •••)

I millimetri di pioggia caduti in quattro stazioni meteo di Milano nei primi sei mesi dell'anno. I valori sono **verosimili, non misure**: servono a fare i conti.

```
mesi = ['gennaio', 'febbraio', 'marzo', 'aprile', 'maggio', 'giugno']

piogge = [['Brera',      58, 49, 71, 83, 102, 66],
          ['Lambrate',   61, 52, 68, 90,  97, 71],
          ['Juvara',     55, 47, 74, 79, 110, 63],
          ['Parco Nord', 64, 55, 70, 88, 105, 74]]
```

Scrivi un programma che stampi:

1. per ogni **stazione**, il totale dei sei mesi;
2. per ogni **mese**, la media delle quattro stazioni;
3. il mese **più piovoso**, cioè quello con la media più alta.

Prima di scrivere il ciclo, guarda la prima riga e chiediti: **da quale colonna cominciano i numeri?**

Soluzione

Mostra la soluzione

```
mesi = ['gennaio', 'febbraio', 'marzo', 'aprile', 'maggio', 'giugno']

piogge = [['Brera',      58, 49, 71, 83, 102, 66],
          ['Lambrate',   61, 52, 68, 90,  97, 71],
          ['Juvara',     55, 47, 74, 79, 110, 63],
          ['Parco Nord', 64, 55, 70, 88, 105, 74]]

primoMese = 1   # la colonna 0 e' il nome della stazione

# 1. somma per riga: una stazione, tutti i mesi
for riga in piogge:
    totale = 0
```

```
    for colonna in range(primoMese, len(riga)):
        totale = totale + riga[colonna]
    print(riga[0], totale)

# 2. somma per colonna: un mese, tutte le stazioni
medie = []
for indiceMese in range(len(mesi)):
    somma = 0
    for riga in piogge:
        somma = somma + riga[indiceMese + primoMese]
    media = somma / len(piogge)
    medie.append(media)
    print(mesi[indiceMese], media)

# 3. il mese con la media piu' alta
piuPiovoso = 0
for indiceMese in range(len(medie)):
    if medie[indiceMese] > medie[piuPiovoso]:
        piuPiovoso = indiceMese
print("il mese piu' piovoso e'", mesi[piuPiovoso])
```

Stampa:

```
Brera 429
Lambrate 439
Juvara 428
Parco Nord 456
gennaio 59.5
febbraio 50.75
marzo 70.75
aprile 85.0
maggio 103.5
giugno 68.5
il mese piu' piovoso e' maggio
```

Per riga e per colonna cambia solo quale ciclo sta fuori. Per il totale di una stazione il ciclo esterno scorre le righe e quello interno le colonne; per la media di un mese il contrario. È lo schema dei frame *Per riga* e *Per colonna* di `L07_g`.

Il salto della colonna del nome ha un nome, `primoMese`. Nella somma per riga si parte da lì; nella somma per colonna il mese di indice `indiceMese` sta nella colonna `indiceMese + primoMese`, perché la lista `mesi` comincia da 0 e la riga no. Scriverlo una volta, con un commento, è quello che mancava alla tabella della Florida: lì il salto era un `2` scritto a mano, ed era sbagliato.

Le medie vanno in una lista, `medie`, perché il punto 3 ne ha bisogno dopo il ciclo. Il mese più piovoso si cerca come il massimo dell'esercizio 11, ma si tiene l'**indice** del massimo e non il valore: è l'indice che porta al nome, `mesi[piuPiovoso]`.